

Autonomous Security Lab Assistant

Complete Product Guide

Purpose

A secure MCP-style backend for authorized cybersecurity lab reconnaissance, evidence collection, reporting, auditability, and AI-agent tool orchestration.

Document Metadata

Field	Value
Project	Autonomous Security Lab Assistant
Version	0.4.0
Workspace	D:\MCP_Demo
Audience	Students, backend engineers, AI engineers, security learners, reviewers
Deliverable	Detailed PDF manual with diagrams

Table of Contents

- Executive Summary
- Who Uses This Product
- Primary Use Case
- What The Product Does
- How To Run It
- MCP-Style Server
- Security Model
- Artifact System
- File-By-File Walkthrough
- Line-By-Line Conceptual Workflow
- Testing
- Example Output
- Non-Goals
- How To Present It
- Future Improvements

1. Executive Summary

The Autonomous Security Lab Assistant is a safe-by-default cybersecurity lab assistant built around MCP-style tool orchestration.

It lets a user or AI agent inspect authorized lab targets, collect structured evidence, generate findings, save reports, export SARIF, index runs, and maintain tamper-evident audit logs.

The project is not a simple chatbot, not a CRUD application, and not a wrapper around one API. It is a security orchestration backend designed to demonstrate AI agents, MCP architecture, defensive cybersecurity workflows, policy enforcement, persistence, reporting, and backend engineering discipline.

Core Principle
Policy first, tools second, evidence always.

2. Who Uses This Product

Cybersecurity students use it to learn safe reconnaissance workflows.

Backend engineers use it to study protocol design, tool registries, persistence, reporting, and secure input validation.

AI engineers use it to understand how agents can call tools through controlled MCP-style interfaces.

Security researchers, CTF players, and teachers use it in owned labs, private ranges, and intentionally vulnerable training environments.

User Groups

User	Why they use it	What they gain
Cybersecurity student	Practice safe lab recon	Scope validation, evidence, reports
Backend engineer	Study product backend design	JSON-RPC, validation, storage, tests
AI engineer	Build safe agent tools	MCP-style schemas and orchestration
Security team	Prototype defensive workflows	Auditability, SARIF, risk scoring
Portfolio builder	Show advanced capability	Security plus backend plus agent architecture

Product Architecture

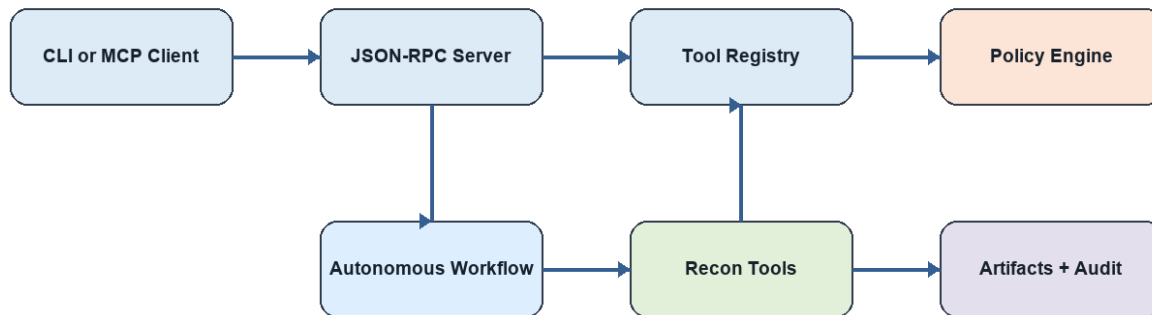


Figure 1. Product architecture.

3. Primary Use Case

The main use case is authorized lab reconnaissance. A user points the assistant at an allowed local or private target, chooses ports, and receives structured results.

For example, a developer may run a local service on 127.0.0.1:8000 and ask the assistant to inspect common web ports. The assistant validates scope, scans only the requested ports, collects HTTP headers, checks TLS metadata when HTTPS exists, checks security.txt and robots.txt, extracts safe HTML page intelligence, generates findings, scores risk, and saves artifacts.

CLI recon example

```
python -m security_lab_assistant.cli recon 127.0.0.1 --ports 80,443,8000,8080
```

Autonomous Recon Workflow

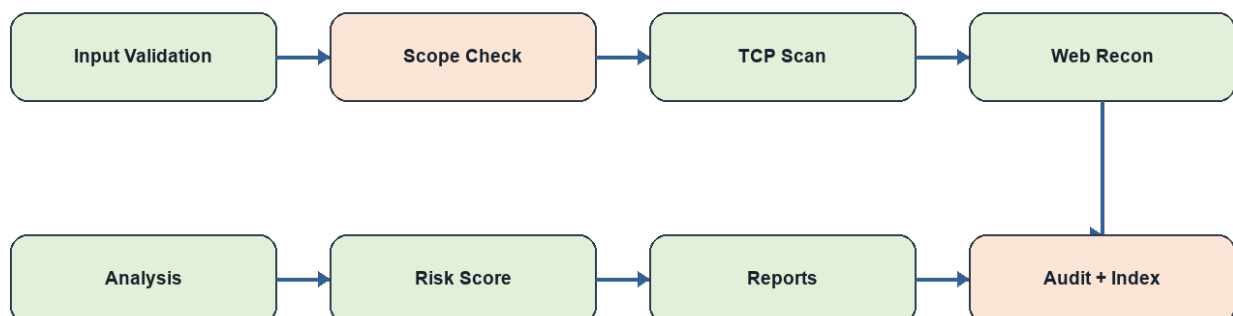


Figure 2. Autonomous recon workflow.

4. What The Product Does

The product exposes narrow security tools. Each tool has a name, description, input schema, and handler function.

Every active network tool checks the policy engine before it touches a target. Unsafe requests produce structured refusals instead of uncontrolled exceptions.

Tool Catalog

Tool	Purpose	Safety controls
scope.validate	Checks whether a target is allowed	Target normalization, CIDR and hostname policy
scan.tcp_connect	Bounded TCP connect scan	Port limit, blocked ports, timeout, worker cap
recon.http_headers	Collect selected HTTP headers	URL policy, no auto-follow redirects
recon.tls_certificate	Inspect TLS certificate metadata	Target and port policy
recon.well_known_security	Check robots.txt and security.txt	In-scope URL checks
recon.web_page_intel	Extract safe HTML indicators	No JavaScript execution
web.fetch_text	Fetch bounded text content	Max byte limit, URL policy
analyze.security_headers	Create header-hardening finding	Structured analysis only
run.list / run.get / run.search	Manage run history	Bounded inputs and UUID-only lookup
run.verify_audit	Verify audit hash chain	Tamper-evidence check
workflow.autonomous_rec on	Run full workflow	All controls combined

5. How To Run It

Open PowerShell in D:\MCP_Demo. The project requires Python 3.11 or newer and currently has no third-party runtime dependencies for the application itself.

Run all tests

```
python -m unittest discover -s tests
```

Run recon

```
python -m security_lab_assistant.cli recon 127.0.0.1 --ports 80,443,8000,8080
```

List runs

```
python -m security_lab_assistant.cli runs
```

Search runs

```
python -m security_lab_assistant.cli search --query 127.0.0.1 --status completed
```

Show one run

```
python -m security_lab_assistant.cli show <run_id>
```

6. How To Use The MCP-Style Server

The server reads newline-delimited JSON-RPC messages from stdin and writes JSON-RPC responses to stdout. Supported methods are initialize, tools/list, tools/call, and notifications/initialized.

Start server

```
python -m security_lab_assistant.server
```

Initialize

```
{"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {"clientInfo": {"name": "demo-client", "version": "0.1.0"}}}
```

List tools

```
{"jsonrpc": "2.0", "id": 2, "method": "tools/list", "params": {}}
```

Run workflow

```
{"jsonrpc": "2.0", "id": 4, "method": "tools/call", "params": {"name": "workflow.autonomous_recon", "arguments": {"target": "127.0.0.1"}}
```

7. Security Model

The product is strict because cybersecurity automation must not act before policy allows it. No software can honestly promise zero vulnerabilities. The professional claim is that this project uses layered controls, regression tests, narrow tool boundaries, and explicit residual-risk documentation.

Security Boundary Model

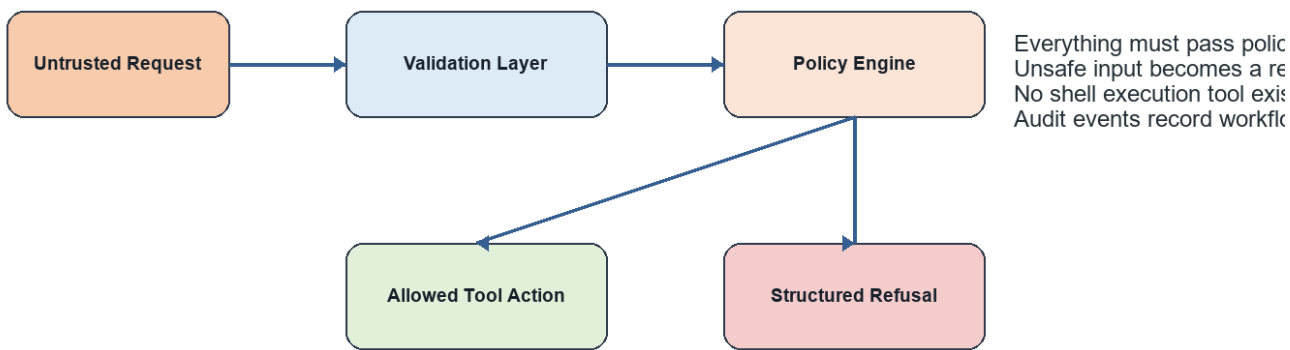


Figure 3. Security boundary model.

Security Controls

Control	What it prevents
Deny-by-default DNS targets	Accidental public-host scans
CIDR allowlist	Out-of-scope IP targets
Hostname allowlist	Unapproved DNS names
Blocked ports	Unsafe or unwanted service probing
Maximum ports per scan	Large unintended scans
URL credential rejection	Credential leakage and ambiguous authority
Fragment and parameter rejection	Ambiguous URL behavior
UUID-only run lookup	Path traversal through run IDs
Project-local artifact root	Writes outside the workspace
Atomic writes	Partial or corrupted artifacts
Hash-chained audit log	Undetected audit tampering
No shell tool	Arbitrary command execution

8. Artifact System

A serious security tool must preserve evidence. This product creates local artifacts for each workflow run. The artifact directory is restricted to the project by policy. Absolute or out-of-project paths are refused.

Artifact System

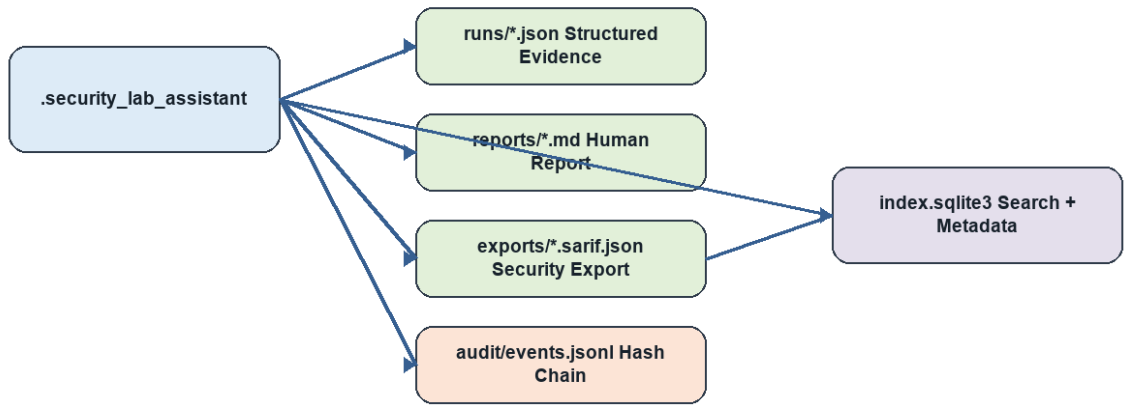


Figure 4. Artifact layout.

Artifacts

Path	Purpose
.security_lab_assistant/runs/.json	Complete structured evidence
.security_lab_assistant/reports/.md	Human-readable Markdown report
.security_lab_assistant/exports/.sarif.json	Security tooling export
.security_lab_assistant/audit/events.jsonl	Tamper-evident audit events
.security_lab_assistant/index.sqlite3	Run and audit index

9. File-By-File Walkthrough

This section explains what each important file does and why it exists.

Important Files

File	Purpose
pyproject.toml	Package metadata, version, Python requirement, CLI entry points.
configs/lab_policy.json	Default security policy: CIDRs, hostnames, blocked ports, scan limits, artifact path.
models.py	Core dataclasses: ToolResult, Finding, RunContext.
policy.py	Security gatekeeper for targets, URLs, ports, and artifact paths.
validation.py	Shared safe parsing for strings, ports, limits, statuses, run IDs, and timeouts.
tools/base.py	ToolSpec abstraction and structured refusal helper.
tools/scope.py	Implements scope.validate.
tools/network.py	TCP scan, HTTP headers, TLS certs, well-known files, fetch, and page intel.
tools/reporting.py	Analyzes security headers and creates findings.
tools/runs.py	Run list, get, search, and audit verification tools.
tools/registry.py	Central catalog of exposed MCP tools and schemas.
workflows/autonomous_recon.py	Main orchestration loop from validation to persistence.
storage.py	Atomic writes, SQLite index, audit hash chain, run persistence.
reporting.py	Markdown report renderer.
risk.py	Risk scoring and band calculation.
sarif.py	SARIF export renderer.
mcp_protocol.py	MCP-style JSON-RPC method handler.
server.py	Stdio JSON-RPC server.
cli.py	Command-line interface for recon and run management.
tests/	Security, protocol, persistence, and feature regression tests.

10. Line-By-Line Conceptual Workflow

The main workflow is in `security_lab_assistant/workflows/autonomous_recon.py`. The table below explains the logic at a practical line-by-line level.

Main Workflow Logic

Step	Code idea	Explanation
1	Read target	require_string rejects empty, overlong, or control-character input.
2	Read objective	Uses provided objective or defaults to baseline web reconnaissance.
3	Parse ports	parse_ports rejects invalid values, booleans, duplicates, and unsafe types.
4	Create RunContext	Stores target, status, phases, evidence, findings, warnings.
5	Audit start	workflow.started is appended to the audit log.
6	Scope phase	scope.validate confirms target is allowed before network activity.
7	Refusal path	If scope fails, status becomes refused and artifacts are still saved.
8	Port scan phase	scan.tcp_connect checks requested ports with bounded concurrency.
9	Service recon phase	Open web ports are converted into HTTP or HTTPS URLs.
10	Header recon	recon.http_headers captures selected headers without following redirects.
11	TLS recon	HTTPS services trigger certificate metadata collection.
12	Well-known checks	robots.txt and security.txt locations are inspected.
13	Page intelligence	HTML structure is parsed safely without executing scripts.
14	Analysis	analyze.security_headers turns observations into findings.
15	Persistence	Risk, Markdown, SARIF, JSON evidence, SQLite index, and audit events are written.
16	Return result	Returns run ID, risk, warnings, findings, steps, and artifact paths.

11. How Testing Works

The tests use Python's standard unittest framework. They verify policy behavior, refusal paths, protocol handling, persistence, SARIF export, risk scoring, audit verification, and security edge cases.

Test Files

Test file	What it verifies
tests/test_policy.py	Default policy, local targets, public IP refusal, port limits
tests/test_tools.py	Tool refusal behavior and workflow stopping
tests/test_mcp_protocol.py	JSON-RPC initialize, tools/list, tools/call validation
tests/test_product_edges.py	Persistence, invalid URLs, DNS denial, path traversal, limits
tests/test_enterprise_features.py	SARIF, risk scoring, SQLite search, audit verification

Expected test result

```
python -m unittest discover -s tests
```

```
Ran 28 tests  
OK
```

12. Example Output Explained

A workflow response contains ok, run_id, target, status, open ports, severity counts, risk information, warnings, evidence steps, findings, and artifact paths.

Simplified output

```
{  
  "ok": true,  
  "data": {  
    "run_id": "44da8f18-1368-425a-85d8-ee1e519e6a2f",  
    "target": "127.0.0.1",  
    "status": "completed",  
    "open_ports": [],  
    "risk": {"score": 2, "band": "low"},  
    "warnings": ["No open ports were detected in the requested scan set."],  
    "run_path": ".security_lab_assistant/runs/<run_id>.json",  
    "report_path": ".security_lab_assistant/reports/<run_id>.md",  
    "sarif_path": ".security_lab_assistant/exports/<run_id>.sarif.json"  
  }  
}
```

13. What This Product Does Not Do

It does not exploit vulnerabilities, brute force passwords, run malware, execute arbitrary shell commands, scan arbitrary public internet targets by default, bypass authentication, perform stealth scanning, or replace professional security testing.

14. How To Present This Project

Long version: Autonomous Security Lab Assistant is a safe-by-default MCP-style security orchestration backend. It validates scope before action, exposes narrow security tools through structured schemas, performs bounded recon against authorized lab targets, persists evidence, generates reports, exports SARIF, maintains a SQLite run index, and records tamper-evident audit logs.

Short version: It is a secure AI-agent tool server for authorized cybersecurity lab reconnaissance.

15. Future Improvements

Add official MCP SDK integration, authenticated HTTP transport, Docker Compose vulnerable lab targets, HTML report export, CI checks, signed releases, file permission hardening, role-based policies, and human approval gates for higher-risk tools.

Final Summary

This project combines AI agents, MCP architecture, cybersecurity guardrails, backend engineering, reporting, persistent evidence, auditability, and tests.